

Laboratorio de Estructura de Computadores

Curso 2023-2024

Práctica 4: Diseño del Microprocesador RISC-V

Ejercicio 1. Preparación de la memoria de programa y datos

En la práctica 3 se escribieron varios programas en ensamblador para RISC-V. Se vio que el simulador RARS, entre otras muchas funciones, permite ver el código máquina de las instrucciones ensamblador escritas en un programa. En este ejercicio se ofrece un programa ensamblador (*ejercicio1.asm*) y unos ficheros VHDL que modelan las memorias de Programa ("*MemProgRISCV.vhd*") y de Datos ("*MemDataRISCV.vhd*") del microprocesador que se está diseñando. Estas memorias tienen un fragmento de los segmentos de código y datos del programa, pero falta por completar algunas instrucciones y datos.

Objetivo

Utilizando la traducción a código máquina que ofrece el simulador RISC-V, se pide completar las memorias de Programa y Datos. Estas memorias se utilizarán para validar el procesador durante el ejercicio 3.

Este ejercicio se valorará en la calificación de evaluación continua si se termina y muestra al profesor durante la primera sesión de prácticas.

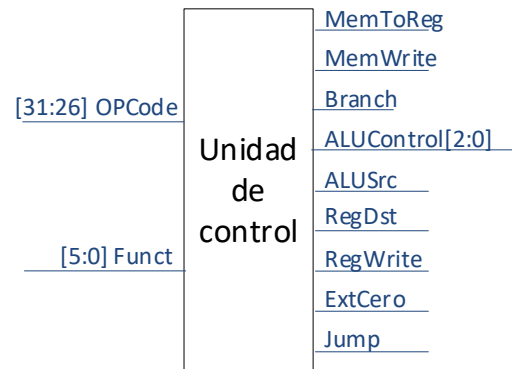
Ejercicio 2a. Diseño de la unidad de control

Diseñar la unidad de control del microprocesador RISC-V uniciclo. La interfaz de este módulo se presenta en la imagen de la derecha.

Este módulo genera 8 señales de control. Estas señales de control definen el comportamiento del resto de elementos que componen el microprocesador.

Como entradas recibe los campos *op*, *funct3* y *funct7* de la instrucción.

En la tabla adjunta se muestran las instrucciones que debe ejecutar el microprocesador:



op	funct3	funct7	Nombre	Operación
0110011	000	0000000	add rd, rs1, rs2	[rd] = [rs1] + [rs2]
0110011	000	0100000	sub rd, rs1, rs2	[rd] = [rs1] - [rs2]
0110011	100	0000000	xor rd, rs1, rs2	[rd] = [rs1] $\oplus$ [rs2]
0110011	110	0000000	or rd, rs1, rs2	[rd] = [rs1] [rs2]
0110011	111	0000000	and rd, rs1, rs2	[rd] = [rs1] & [rs2]
0110011	010	0000000	slt rd, rs1, rs2	rd = (rs1 < rs2)
0000011	010	-	lw rd, imm(rs1)	MEM ([rs1]+SignExt) => [rd]
0010011	000	-	addi rd, rs1, imm	rd = rs1 + SignExt(imm)
0010011	010	-	slti rd, rs1, imm	rd = (rs1 < SignExt(imm))
0010011	100	-	xori rd, rs1, imm	rd = rs1 ^ SignExt(imm)
0010011	110	-	ori rd, rs1, imm	rd = rs1 SignExt(imm)
0010011	111	-	andi rd, rs1, imm	rd = rs1 & SignExt(imm)
1100111	000	-	jalr rd, rs1, imm	PC = rs1 + SignExt(imm) rd = PC + 4
0100011	010	-	sw rs2, imm(rs1)	[rs2] => MEM ([rs1]+SignExt)
1100011	000	-	beq rs1, rs2, label	if (rs1 == rs2) PC = BTA
0010111	-	-	auipc rd, upimm	rd = {upimm, 12'b0} + PC
1101111	-	-	jal rd, label	PC = JTA, rd = PC + 4

Para realizar la instrucción auipc (no vista en la unidad de teoría), el dato inmediato de 20 bits (31:12) se convierte a 32 bits, y se le suma el PC. El resultado de esta operación llega al registro rd a través del multiplexor controlado por ResultSrc, ya que esta señal de control valdrá "11" en este caso.

Objetivo

Comprender el problema propuesto, y completar la tabla de control utilizando el esquemático que se encuentra en la última página del enunciado.

Este ejercicio se valorará en la calificación de evaluación continua si se termina y muestra al profesor durante la primera sesión de prácticas.

INSTRUCCIÓN	ResultSrc	MemWrite	Branch	PCSrc	ALUControl	ALUSrc	ImmSrc	RegWrite	Jump	Jalr
R-Type										
lw										
sw										
beq										
jal										
jalr										
auipc										
imm										

Ejercicio 2b. Implementación en VHDL de la unidad de control

Utilizando la tabla del anterior ejercicio, diseñar en VHDL la unidad de control del microprocesador RISC-V unicycle.

Objetivo

Comprender el problema propuesto. Plantear e implementar una solución para dicho problema, para ello debe crear desde cero, un archivo denominado UnidadControl.vhd, es decir debe diseñar las librerías, la entidad y la arquitectura. Como apoyo en el diseño del archivo de la unidad de control y para probar su funcionalidad se le facilita el archivo con el banco de pruebas “UnidadControlTb.vhd”.

Este ejercicio se valorará en la calificación de evaluación continua si se termina y muestra al profesor durante la segunda sesión de prácticas.

Ejercicio 3. Diseño completo del microprocesador

En este ejercicio se deberá realizar el diseño completo del microprocesador. Para llevar a cabo este ejercicio se deberán incluir los módulos de la ALU (ALURISCV.vhd) y del banco de registros (RegsRISCV.vhd) de la práctica 2.

El sistema que se está desarrollando sigue la arquitectura Harvard, con una memoria para el código y otra para los datos. Las entradas del microprocesador “MicroRISCV” son una señal de *Reset* activa a nivel bajo (NRst), el reloj (Clk), la entrada de memoria de programa (MemProgData) y la entrada de memoria de datos (MemDataDataRead). Como salidas, la señal que habilita la escritura en memoria de datos (MemDataWe), la dirección de memoria de programa (MemProgAddr), la dirección de memoria de datos (MemDataAddr) y el bus de escritura en memoria (MemDataDataWrite).

Al final de este enunciado se puede encontrar un esquemático correspondiente al microprocesador que debe implementarse. Este esquemático corresponde a las transparencias de la unidad 4, aunque ya están añadidos los elementos lógicos necesarios para implementar las instrucciones que no figuran en dicha unidad.

Validación

Para realizar las pruebas del microprocesador se deben usar las memorias de programa y de datos del ejercicio 1. La memoria de programa ("*MemProgRISCV.vhd*") describe una memoria ROM que recoge las instrucciones del programa. La memoria de datos ("*MemDataRISCV.vhd*") es una memoria de lectura asíncrona y escritura síncrona en flanco de subida. El testbench que se suministra ("*MicroRISCVTb.vhd*") instancia la CPU a diseñar por el alumno en el ejercicio 3 y las memorias completadas en el ejercicio 1.

Nota: En la corrección de la práctica, para poder utilizar el archivo **.do** facilitado, al instanciar el Banco de Registros en el archivo *MicroRISCV.vhd*, es necesario que el nombre de dicha instancia sea GPR [GPR: RegsRISCV port map (...)].

En este ejercicio se debe ejecutar el programa y comprobar que el resultado obtenido en el banco de registros del microprocesador diseñado, es el mismo que el que se obtiene utilizando el simulador RARS cuando se ejecuta el fichero en ensamblador *Ejercicio1.asm*. Para comprobar el correcto funcionamiento del micro se sugiere utilizar las formas de onda incluidas en "*Wave_Ejercicio3.do*" (script para la visualización de señales y el banco de registros en *ModelSim*), añadiendo las señales que se considere oportuno.

Objetivo

Comprender el problema propuesto. Plantear e implementar una solución para dicho problema. Ejecutar el banco de pruebas con las memorias entregadas para corregir los errores cometidos durante la fase de diseño.

Este ejercicio se valorará en la calificación de evaluación continua si se termina y muestra al profesor durante la tercera sesión de prácticas.

